

# C# - Phần 9.9: Publisher / Subscriber

Lý thuyết khái niệm -> Code -> Phân tích luồng -> Bài tập

**Mục tiêu:** Phân biệt rõ Publisher, Subscriber, Event và Event Handler; hiểu vì sao Publisher không cần biết cụ thể Subscriber nào đang nhận Event.

## I. Lý thuyết khái niệm

### 1. Publisher / Subscriber là gì?

Publisher / Subscriber là cách tổ chức để các object giao tiếp với nhau thông qua Event.

Một object phát thông báo khi một sự việc xảy ra. Những object khác nếu quan tâm thì đăng ký để nhận thông báo đó.

```
Một sự việc xảy ra
↓
Publisher phát Event
↓
Subscriber nhận thông báo
↓
Event Handler được gọi
```

### 2. Publisher là gì?

Publisher là class hoặc object sở hữu Event và quyết định khi nào Event được phát.

- Publisher sở hữu Event.
- Publisher thực hiện nghiệp vụ của chính nó.
- Khi sự việc phù hợp xảy ra, Publisher phát Event bằng Invoke().

```
OrderService
→ Publisher

OrderCompleted
→ Event

OrderService quyết định khi nào:
OrderCompleted?.Invoke(orderId);
```

### 3. Subscriber là gì?

Subscriber là class hoặc object quan tâm đến Event và đăng ký một method phù hợp vào Event đó.

```
EmailService emailService = new EmailService();

orderService.OrderCompleted +=
    emailService.SendEmail;
```

Trong ví dụ trên, emailService là Subscriber.

### 4. Event Handler là gì?

Event Handler là method được đăng ký vào Event và sẽ được gọi khi Event xảy ra.

```
emailService
→ Subscriber
```

```
emailService.SendEmail  
→ Event Handler
```

**Điểm cần nhớ:** Một method có chữ ký phù hợp chưa tự động là Event Handler. Nó trở thành Event Handler khi được đăng ký vào Event bằng +=.

## 5. Handler phải khớp chữ ký Event

```
public event Action<string>? OrderCompleted;
```

Action<string> yêu cầu handler nhận 1 string và trả void.

```
public void SendEmail(string orderId)  
{  
    Console.WriteLine(orderId);  
}
```

Method trên phù hợp và có thể đăng ký làm Event Handler.

## 6. Publisher không cần biết Subscriber cụ thể

Đây là ý quan trọng nhất của bài.

Publisher chỉ cần sở hữu Event và phát Event. Nó không cần biết EmailService, LogService, SmsService hay bất kỳ Subscriber cụ thể nào có tồn tại hay không.

```
public event Action<string>? OrderCompleted;  
  
...  
  
OrderCompleted?.Invoke(orderId);
```

Ai muốn nhận thông báo thì tự đăng ký từ bên ngoài.

**Lợi ích:** Cách tổ chức này giúp giảm sự phụ thuộc giữa các class. Muốn thêm Subscriber mới thường không cần sửa Publisher.

## 7. Event và Publisher / Subscriber có phải một khái niệm không?

Không.

- Event là cơ chế/công cụ của C#: event, +=, -=, ?.Invoke().
- Publisher / Subscriber là cách tổ chức vai trò giữa các object khi sử dụng Event.

```
Event  
→ công cụ
```

```
Publisher / Subscriber  
→ cách tổ chức các object sử dụng công cụ đó
```

## 8. Quy tắc Event vẫn được giữ nguyên

- Publisher sở hữu Event.
- Subscriber đăng ký bằng +=.
- Subscriber hủy đăng ký bằng -=.
- Code bên ngoài không được ghi đè Event bằng =.
- Code bên ngoài không được tự Invoke Event.
- Publisher quyết định thời điểm Invoke Event.

## II. Ví dụ code hoàn chỉnh

### 1. Publisher - OrderService

```
public class OrderService
{
    public event Action<string>? OrderCompleted;

    public void CompleteOrder(string orderId)
    {
        if (string.IsNullOrEmpty(orderId))
        {
            throw new ArgumentException(
                "Order ID cannot be null, empty, or whitespace.",
                nameof(orderId));
        }

        string normalizedOrderId =
            orderId.Trim();

        Console.WriteLine(
            $"Order {normalizedOrderId} completed.");

        OrderCompleted?.Invoke(
            normalizedOrderId);
    }
}
```

OrderService là Publisher vì nó sở hữu OrderCompleted và quyết định khi nào phát Event.

### 2. Subscriber - EmailService

```
public class EmailService
{
    public void SendEmail(string orderId)
    {
        Console.WriteLine(
            $"Email sent for {orderId}.");
    }
}
```

EmailService object là Subscriber; SendEmail() là Event Handler sau khi được đăng ký.

### 3. Subscriber - LogService

```
public class LogService
{
    public void WriteLog(string orderId)
    {
        Console.WriteLine(
            $"Log written for {orderId}.");
    }
}
```

### 4. Subscriber - NotificationService

```
public class NotificationService
{
    public void ShowNotification(string orderId)
```

```

    {
        Console.WriteLine(
            $"Notification shown for {orderId}.");
    }
}

```

## 5. Program - đăng ký và hủy đăng ký

```

public class Program
{
    public static void Main(string[] args)
    {
        OrderService orderService =
            new OrderService();

        EmailService emailService =
            new EmailService();

        LogService logService =
            new LogService();

        NotificationService notificationService =
            new NotificationService();

        orderService.OrderCompleted +=
            emailService.SendEmail;

        orderService.OrderCompleted +=
            logService.WriteLog;

        orderService.CompleteOrder("DH01");

        orderService.OrderCompleted +=
            notificationService.ShowNotification;

        orderService.CompleteOrder("DH02");

        orderService.OrderCompleted -=
            emailService.SendEmail;

        orderService.CompleteOrder("DH03");
    }
}

```

## III. Phân tích luồng chạy

### 1. Khi DH01 hoàn thành

```

OrderCompleted
├─ SendEmail
└─ WriteLog

```

CompleteOrder("DH01") gọi OrderCompleted?.Invoke("DH01"), nên SendEmail và WriteLog chạy.

## 2. Khi DH02 hoàn thành

```
OrderCompleted
├── SendEmail
├── WriteLog
└── ShowNotification
```

Sau khi đăng ký thêm ShowNotification, cả 3 handler chạy.

## 3. Khi DH03 hoàn thành

```
OrderCompleted
├── WriteLog
└── ShowNotification
```

SendEmail đã được hủy bằng -= nên không còn chạy.

## IV. Hệ thống kiến thức cần nhớ

```
Publisher
→ sở hữu Event
→ quyết định khi nào Event xảy ra
→ Invoke Event

Subscriber
→ quan tâm đến Event
→ đăng ký bằng +=
→ hủy đăng ký bằng -=

Event Handler
→ method được đăng ký
→ được gọi khi Event xảy ra
```

Ví dụ:

```
OrderService
→ Publisher

OrderCompleted
→ Event

EmailService
→ Subscriber

SendEmail()
→ Event Handler
```

## V. Bài tập 9.9 - Publisher / Subscriber

Tạo hệ thống Student Registration:

```
StudentRegistration/
  StudentService.cs
  EmailService.cs
  LogService.cs
  WelcomeNotificationService.cs
  Program.cs
```

StudentService là Publisher.

```
public event Action<string>? StudentRegistered;
```

Tạo method:

```
public void RegisterStudent(string studentName)
```

RegisterStudent() phải:

- Kiểm tra studentName null / empty / whitespace.
- Trim studentName.
- In: Student <name> registered successfully.
- Phát StudentRegistered bằng ?.Invoke().

Tạo 3 Subscriber và handler:

```
EmailService
→ SendWelcomeEmail(string studentName)

LogService
→ WriteRegistrationLog(string studentName)

WelcomeNotificationService
→ ShowWelcomeMessage(string studentName)
```

Trong Program thực hiện:

1. Tạo StudentService.
2. Tạo EmailService.
3. Tạo LogService.
4. Tạo WelcomeNotificationService.
5. Đăng ký SendWelcomeEmail.
6. Đăng ký WriteRegistrationLog.
7. RegisterStudent("Nguyen Van A").
8. Đăng ký thêm ShowWelcomeMessage.
9. RegisterStudent("Tran Thi B").
10. Hủy đăng ký SendWelcomeEmail.
11. RegisterStudent("Le Van C").

## VI. Câu hỏi cuối bài

1. Trong bài này class nào là Publisher? Vì sao?
2. Những object nào là Subscriber?
3. SendWelcomeEmail() là Subscriber hay Event Handler?
4. StudentService có cần biết cụ thể EmailService và LogService không? Vì sao?
5. Nếu muốn thêm SmsService, có cần sửa StudentService không?
6. Event và Publisher/Subscriber có phải cùng một khái niệm không?

## VII. Đáp án lý thuyết chuẩn sau khi làm bài

1. StudentService là Publisher vì nó sở hữu StudentRegistered và quyết định khi nào phát Event bằng StudentRegistered?.Invoke(...).
2. emailService, logService và welcomeNotificationService là các Subscriber vì chúng đăng ký handler vào StudentRegistered.

3. SendWelcomeEmail() là Event Handler; emailService mới là Subscriber.
4. StudentService không cần biết cụ thể EmailService hay LogService. Nó chỉ phát Event, còn Subscriber tự đăng ký từ bên ngoài.
5. Không cần sửa StudentService khi thêm SmsService. Chỉ cần tạo SmsService, tạo handler phù hợp và đăng ký bằng +=.
6. Event là cơ chế/công cụ của C#; Publisher/Subscriber là cách tổ chức các object giao tiếp thông qua Event.

**Mục tiêu đạt bài 9.9:** Nhớ được câu: Publisher chỉ phát Event; Subscriber nào quan tâm thì tự đăng ký; method được đăng ký là Event Handler.